

→ Constructors in Java :-

A constructor is a special block in a class that is used to **initialize an object**.

It is **automatically called** when an **object is created** using the **new** keyword.

Eg;

```
Student s1 = new Student();
```

When this statement executes, the constructor of Student is **called automatically**.

Definition:

A constructor is a special member of class that is automatically invoked when an object is created and ~~maining~~ mainly used to initialize the object data members.

→ Why do we need Constructor?
 Suppose we create an object:

```
Student s1 = new Student();  
[ s1.name = "Akash";  
  s1.age = 21; ]
```

we have to initialize the values separately.

A constructor allows us to initialize them **at the time of object creation**.

Student s1 = new Student();

The constructor can automatically set:

name = Akash

age = 21

So constructors make object initialization easier.

→ Syntax of constructor:

```
class className {
```

```
    className() {
```

```
        // initialization
```

```
    }
```

```
}
```

Eg;

```
class Student {
```

```
    Student() {
```

```
        System.out.println("Constructor called");
```

```
    }
```

```
}
```

Creating object:

```
Student s1 = new Student();
```

output:

Constructor called

→ Important rules of Constructors :-

Ret Rule 1: Constructor name must be the same as the class name :

```
class Student {
    Student() {
    }
}
```

Rule 2: Constructor has no return type :

```
Student() {
}
```

Rule 3: Constructor is called automatically :

```
Student s1 = new Student();
```

We don't normally call it separately.

Rule 4: Constructor can have parameters

```
Student(int age) {
    this.age = age;
}
```


Rule 5: Constructor can be overloaded:

A class can have multiple constructor with different parameter list.

```
Student() {  
}
```

```
Student(int age) {  
}
```

Eg: Simple Constructor

```
class Student {  
    int age;  
    String name;
```

```
    Student() {  
        age = 21;  
        name = "Akash";
```

```
    }
```

```
    void display() {  
        sout("name=" + name);  
        sout("age=" + age);
```

```
    }
```

```
}
```

```
public class prog0 {  
    psum {
```

```
        Student s1 = new Student(); // constructor called  
        s1.display(); } }
```


→ Types of Constructor :-

(i) No-Argument Constructor:

A constructor that does not take any parameter is called a noargument constructor.

Eg;

```
class Student {
```

```
    int age;
```

```
    String name;
```

```
    Student() {
```

```
        age = 21;
```

```
        name = "Akash";
```

```
    }
```

```
}
```

(ii) Default constructor:

If we do not write any constructor in a class, Java automatically provides a constructor called the default constructor.

Eg;

```
class Student {
```

```
    String name;
```

```
    int age;
```

```
}
```


we do not write any constructor.

Java provides a default constructor automatically when an object is created.

```
student s1 = new Student();
```

The instance variable receives their **default values**.

String → null
int → 0

name → null
int = 0

Note:

If we write **any constructor ourselves**, Java does **not** automatically provide the default constructor.

```
Eg: class Student {  
    Student(int age) {  
        sout(age);  
    }  
}
```

Now,

```
student s = new Student();
```

Error, because there is no no-argument constructor.

we must write one ourself:

```
Student() {  
}
```

if we want to use ~~new~~ new Student();

(iii) Parameterized Constructor :-

A constructor that accepts parameter constructors.

eg;

```
class Student {  
    int age;  
    String name;  
}
```

```
Student(int a, String b) {  
    age = a;  
    name = b;  
}
```

```
void display() {  
    sout(name);  
    sout(age);  
}
```

```
}
```

```
public class pogo {  
    prgm {
```

```
Student s1 = new Student(21, "Akash");  
s1.display();  
}
```


→ Parametrized Constructor with **this** :

We use **this** when constructor parameter have the same name as instance variable.

Eg;

```
class Student {
```

```
    int age;
```

```
    String name;
```

```
    Student(int age, String name) {
```

```
        this.age = age;
```

```
        this.name = name;
```

```
    }
```

```
    void display() {
```

```
        System.out.println("name" + name);
```

```
        System.out.println("age" + age);
```

```
    }
```

```
public class Prgo {
```

```
    psum {
```

```
        Student s1 = new Student(21, "Akash");
```

```
        s1.display();
```

```
    }
```

```
}
```


meaning of :

`this.age = age;`

The left side:

`this.age`
 means the **instance variable**.

The right side:

`age`
 means the **constructor parameter**.

`this.age = age`
 ↓ ↓
 object parameter
 variable

→ Constructor Overloading :

When a class has more than one constructor with different parameter list, it is called constructor overloading.

Eg; Class calci {

int a;

int b;

calci() {

a = 0

b = 0

}

calci(int a, int b) {

this.a = a;

this.b = b;

}

void display() {

cout(a);

cout(b);

}

}

public class pogo {

psvm {

Student s1 = new Student();

Student s2 = new Student(10, 20);

s1.display();

s2.display(); }

Akash